

Cpp Other Langs

C++ 与其他常用语言的区别

本文介绍 C++ 与其他常用语言的区别，重点介绍 C 与 C++ 之间重要的或者容易忽略的区别。尽管 C++ 几乎是 C 的超集，C/C++ 代码混用一般也没什么问题，但是了解 C/C++ 间比较重要的区别可以避免碰到一些奇怪的 bug。如果你是以 C 为主力语言的 Oler，那么本文也能让你更顺利地上手 C++。C++ 相比 C 增加的独特特性可以阅读 [C++ 进阶](#) 部分的教程。此外，本文也简要介绍了 Python, Java 和 C++ 的区别。

C 与 C++ 的区别

宏与模板

C++ 的模板在设计之初的一个用途就是用来替换宏定义。学会模板编程是从 C 迈向 C++ 的重要一步。模板不同于宏的文字替换，在编译时会得到更全面的编译器检查，便于编写更健全的代码。模板特性在 C++11 后支持了可变长度的模板参数表，可以用来替代 C 中的可变长度函数并保证类型安全。

指针与引用

C++ 中你仍然可以使用 C 风格的指针，但是对于变量传递而言，更推荐使用 C++ 的[引用](#)特性来实现类似的功能。由于引用指向的对象不能为空，因此可以避免一些空地址访问的问题。不过指针由于其灵活性，也仍然有其用武之地。值得一提的是，C 中的 NULL 空指针在 C++11 起有类型安全的替代品 `nullptr`。引用和指针之间可以通过 [* 和 & 运算符](#) 相互转换。

bool

另请参阅 [布尔类型](#)。

与 C++ 不同的是，C 语言最初并没有布尔类型。

C99 标准加入了 `_Bool` 关键字（以及等效的 `bool` 宏）以及 `true` 和 `false` 两个宏。如果需要使用 `bool`, `true`, `false` 这三个宏，需要在程序中引入 `stdbool.h` 头文件。而使用 `_Bool` 则不需要引入任何额外头文件。

```
1 bool x = true; // 需要引入 stdbool.h
2 _Bool x = 1;   // 不需要引入 stdbool.h
```

C23 起，`true`, `false` 和 `bool` 成为 C 语言中的关键字，使用它们不需要再引入 `stdbool.h` 头文件，同时保留 `_Bool` 作为 `bool` 的替代拼写形式²。

下表展示了 C 语言不同标准下，`bool` 类型支持的变化情况（作为对照，加入了 C++ 的支持情况）：

语言标准	<code>bool</code>	<code>true/false</code>	<code>_Bool</code>
C89	/	/	保留 ³
C99 起, C23 以前	宏, 与 <code>_Bool</code> 等价, 需要引入 <code>stdbool.h</code> 头文件	宏, <code>true</code> 与 1 等价, <code>false</code> 与 0 等价, 需要引入 <code>stdbool.h</code> 头文件	关键字
C23 起	关键字	关键字	关键字 <code>bool</code> 的替代拼写形式
C++	关键字	关键字	保留 ³

struct

尽管在 C 和 C++ 中都有 struct 的概念，但是他们对应的东西是不能混用的！C 中的 struct 用来描述一种固定的内存组织结构，而 C++ 中的 struct 就是一种类，它与类唯一的区别就是它的成员和继承行为默认是 **public** 的，而一般类的默认成员是 **private** 的。这一点在写 C/C++ 混合代码时尤其致命。

另外，声明 struct 时 C++ 也不需要像 C 那么繁琐，C 版本：

```
1 typedef struct Node_t {
2     struct Node_t *next;
3     int key;
4 } Node;
```

C++ 版本

```
1 struct Node {
2     Node *next;
3     int key;
4 };
```

const

const 在 C 中只有限定变量不能修改的功能，而在 C++ 中，由于大量新特性的出现，const 也被赋予的更多用法。C 中的 const 在 C++ 中的继任者是 constexpr，而 C++ 中的 const 的用法请参见 [常值](#) 页面的说明。

内存分配

C++ 中新增了 new 和 delete 关键字用来在「自由存储区」上分配空间，这个自由存储区可以是堆也可以是静态存储区，他们是为了配合「类」而出现的。其中 delete[] 还能够直接释放动态数组的内存，非常方便。new 和 delete 关键字会调用类型的构造函数和析构函数，相比 C 中的 malloc()、realloc()、free() 函数，他们对类型有更完善的支持，但是效率不如 C 中的这些函数。

简而言之，如果你需要动态分配内存的对象是基础类型或他们的数组，那么你可以使用 malloc() 进行更高效的内存分配；但如果你新建的对象是非基础的类型，那么建议使用 new 以获得安全性检查。值得注意的是尽管 new 和 malloc() 都是返回指针，但是 new 出来的指针 只能用 delete 回收，而 malloc() 出来的指针也只能用 free() 回收，否则会有内存泄漏的风险。

变量声明

C99 前，C 的变量声明必须位于语句块开头，C++ 和 C99 后无此限制。

可变长数组

C99 后 C 语言支持 VLA（可变长数组），C++ 始终不支持。

结构体初始化

C99 后 C 语言支持结构体的 [指派符初始化](#)（但是在 C11 中为可选特性），C++ 直到 C++20 才支持有顺序要求的指派符初始化，且 C 语言支持的乱序、嵌套、与普通初始化器混用、数组的指派符初始化特性 C++ 都不支持¹。

注释语法

C++ 风格单行注释 //，C 于 C99 前不支持。

Python 与 C++ 的区别

Python 是目前机器学习界最常用的语言。相比于 C++，Python 的优势在于易于学习，易于实践。Python 有着更加简

单直接的语法，比如在定义变量时，不需要提前声明变量类型。但是，这样的简单也是有代价的。Python 相比于 C++ 牺牲了性能。C++ 几乎适用于包括嵌入式系统的所有平台，并且有着更快的执行速度，但是 Python 只可以在某些支持高级语言的平台上使用。C++ 更接近底层，所以可以用来进行编写操作系统。

Java 与 C++ 的区别

Java 与 C++ 都是面向对象的语言，都使用了面向对象的思想（封装、继承、多态），由于面向对象有许多非常好的特性（继承、组合等），因此二者有很好的可重用性。所以相比于 Python，Java 和 C++ 更加类似。

二者最大的区别在于 Java 有 JVM 的机制。JVM 全称是 Java Virtual Machine，中文意为 Java 虚拟机。Java 语言的一个非常重要的特点就是与平台的无关性。而使用 Java 虚拟机是实现这一特点的关键。一般的高级语言如果要在不同的平台上运行，至少需要编译成不同的目标代码。而引入 Java 语言虚拟机后，Java 语言在不同平台上运行时不需要重新编译。Java 语言使用 Java 虚拟机屏蔽了与具体平台相关的信息，使得 Java 语言编译程序只需生成在 Java 虚拟机上运行的目标代码（字节码），就可以在多种平台上不加修改地运行。

因为这个特点，Java 经常被用于需要移植到不同平台程序的开发。但是也由于编译 Java 程序时需要从字节码开始，所以 Java 的性能没有 C++ 好。

参考资料

-
1. https://en.cppreference.com/w/cpp/language/aggregate_initialization ↩
 2. <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3054.pdf>。 ↩
 3. C 和 C++ 均规定，以一个下划线跟着一个大写字母开头的标识符是被保留的，详见 <https://en.cppreference.com/w/c/language/identifier>。 ↩↩
-

本页面最近更新：2024/4/19 14:58:08，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[cmpute](#), [StudyingFather](#), [CCXX XI](#), [CoelacanthusHex](#), [Enter-tainer](#), [lr1d](#), [ksyx](#), [NachtgeistW](#), [ouuan](#), [Persdre](#), [Pologue](#), [Suyun514](#), [Tiphereth-A](#), [TRSWNCA](#), [Xeonacid](#)

本页面的全部内容[在 CC BY-SA 4.0 和 SATA 协议之条款下](#)提供，附加条款亦可能应用